Faculty of Computing and Artificial Intelligence

Cairo University

CS241/SCS241: Operating Systems

Assignment 1: Hospital Emergency Room Simulation

Assignment Description

In this assignment, you are required to implement a classic **Producer–Consumer Problem** using the **Bounded Buffer** pattern to simulate a busy **Hospital Emergency Room (ER)**. The simulation should demonstrate proper synchronization between multiple patient arrivals and available doctors while preventing race conditions and ensuring correct access to shared resources.

The simulation models an emergency department with a **limited waiting room capacity**, a **fixed number of treatment rooms**, and a **fixed number of doctors** available to treat patients concurrently.

System Requirements

- Each **patient** arriving at the hospital attempts to enter the waiting room.
- The **waiting room** has a **limited capacity** specified by the user.
- If the waiting room is full, newly arriving patients must wait outside the room until a seat becomes available.
- **Doctors** treat patients concurrently. A doctor remains idle until a patient is available in the waiting room. When a patient is available, a doctor is assigned to this patient for treatment.
- Before beginning the treatment of a patient, a doctor must try to acquire an available **treatment room** (only **two treatment rooms exist** in this whole emergency department)
- If the doctor is unable to acquire a treatment room within a **specific period of time**, the doctor sends the patient back to the waiting room. Otherwise, the treatment of the patient starts and takes some time.
- Once the treatment is completed, the doctor releases the treatment room, potentially allowing another doctor to acquire it and a patient to be treated, and so on.

The output must clearly indicate all system activities whenever a patient:

- Arrives at the hospital
- Enters the waiting room
- Waits outside because the waiting room is full
- Is assigned to a doctor
- Begins treatment in a treatment room acquired by his doctor

- Goes back to the waiting room if his doctor could not acquire a treatment room in time
- Finishes treatment
- Leaves the hospital

Implementation Requirements

You must implement the following classes in **Java**, but **you will also need additional classes** to create and manage shared resources.

1. Hospital

Responsible for:

- Reading user input:
    - Waiting room capacity
    - Number of doctors
    - Number of patients
- Creating doctors and patients
- Displaying all simulation events

---

2. Semaphore

Implement the semaphore class **as described in the labs** and use it in this assignment as you see fit. You should also **add a custom method** in this class that attempts to acquire a resource within a given time limit (i.e., this method returns false if no resource became available within the given time)

---

3. Patient

Each patient should:

1. Arrive
2. Wait outside if the waiting room is full, or
3. Enter the waiting room

---

4. Doctor

Each doctor should:

1. Remain idle until a patient is available
2. Take one patient from the waiting room and become assigned to this patient
3. Try to acquire a treatment room within **three seconds**

4. Begin treatment if the room is acquired, otherwise send the patient back to the waiting room
5. Simulate treatment for a **random** period of time
6. Finish treatment
7. Become available for another patient

Synchronization Requirements

Your implementation **must** use:

- Semaphores
- Bounded Buffer
- Producer–Consumer synchronization

The waiting room and the treatment rooms are separate shared resources and should be synchronized independently. These resources must always remain race-condition-free. The synchronization should also ensure no deadlocks occur.

Sample Input

Waiting room capacity: 2
Number of doctors: 3
Number of patients: 5

Sample Output *(one of the possible scenarios)*

Patients arriving:
P1 P2 P3 P4 P5

P1 arrived.
P1 entered the waiting room.

P2 arrived.
P2 entered the waiting room.

Doctor 1 is assigned to P1.
Doctor 1 is trying to acquire a treatment room.

Doctor 2 is assigned to P2.

Doctor 1 acquired a treatment room.
Doctor 1 begins treating P1.

Doctor 2 is trying to acquire a treatment room.
Doctor 2 acquired a treatment room.
Doctor 2 begins treating P2.

P3 arrived.
P3 entered the waiting room.

P4 arrived.
P4 entered the waiting room.

P5 arrived.
The waiting room is full.
P5 is waiting outside for an available seat.

Doctor 3 is assigned to P3.
Doctor 3 is trying to acquire a treatment room.

P5 entered the waiting room.

Doctor 3 could not acquire a treatment room.

Doctor 3 is sending P3 back to the waiting room.
The waiting room is full.
P3 is waiting outside for an available seat.

Doctor 1 finished treating P1.
Doctor 1 released the treatment room.
Doctor 1 is now available.

P1 left.

Doctor 1 is assigned to P4.

P3 entered the waiting room.

Doctor 3 is now available.
Doctor 3 is assigned to P5.
Doctor 3 is trying to acquire a treatment room.
Doctor 3 acquired a treatment room.
Doctor 3 begins treating P5.

Doctor 1 is trying to acquire a treatment room.

Doctor 2 finished treating P2.
Doctor 2 released the treatment room.
Doctor 2 is now available.

Doctor 1 acquired a treatment room.
Doctor 1 begins treating P4.

P2 left.

Doctor 2 is assigned to P3.
Doctor 2 is trying to acquire a treatment room.

Doctor 1 finished treating P4.
Doctor 1 released the treatment room.

Doctor 2 acquired a treatment room.
Doctor 2 begins treating P3.

Doctor 1 is now available.

P4 left.

Doctor 3 finished treating P5.
Doctor 3 released the treatment room.
Doctor 3 is now available.

P5 left.

Doctor 2 finished treating P3.
Doctor 2 released the treatment room.
Doctor 2 is now available.

P3 left.

All patients have been treated.
Simulation completed.

Assignment Instructions

Please read these notes carefully to avoid losing grades:

1. Teams must consist of exactly **4 or 5** students from the **same lab** group or with the same TA.
2. Submit **one ZIP file** with the naming convention: **A1_TAname_Group_ID1_ID2_ID3_ID4**
3. Only **one** team member should turn in the assignment. Upload the **final** version only.
4. The **deadline** will be announced on Classroom and **no late** submission is allowed.
5. Each violation of the instructions or requirements will result in the **deduction** of 1 mark.
6. All team members must **understand all parts** of the code.
7. **Cheaters** will receive a **negative** grade and no excuses will be accepted as per the "**Plagiarism Scope**" document.

Grading Criteria

Component	Marks
Hospital	1
Semaphore	1
Patient	0.5
Doctor	1.5
Additional class(es) for shared resources and their logic	1.5
Correct synchronization logic and output	1.5
